

Computational Physics_ Dr. Ejtehad

Homework01_Fractals

Mahsa Rahimi_99100671

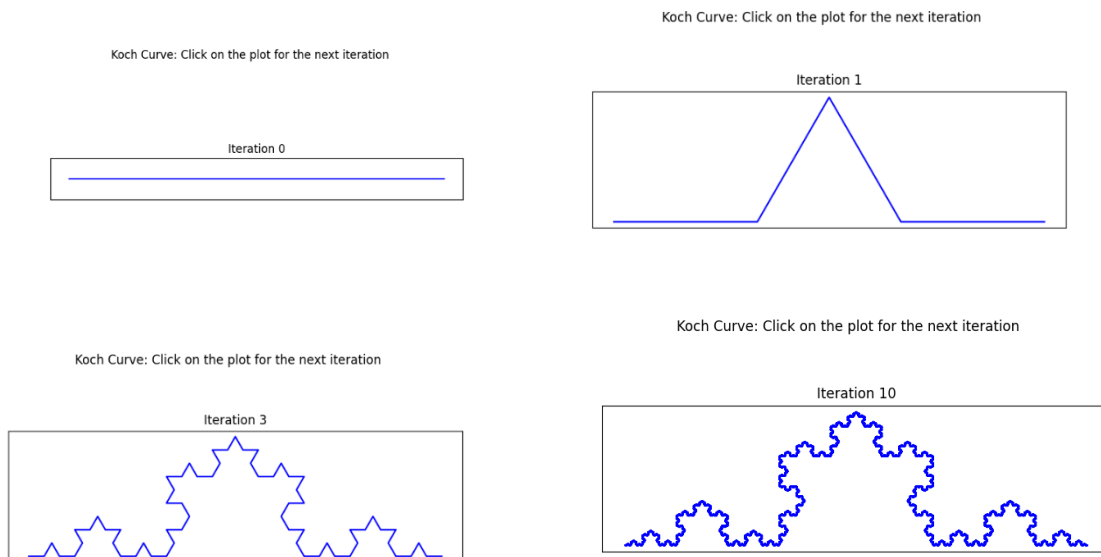
1- Koch Curve

General Idea: I implemented an iterative approach to construct the Koch curve, a well-known fractal. The Koch curve is generated by recursively modifying a straight-line segment. At each iteration, every segment is divided into three equal parts, with the middle segment replaced by two new segments forming an equilateral triangle without the base. This process introduces self-similarity, a key characteristic of fractals.

Algorithm and Approach:

- The curve starts as a simple line segment from (0,0) to (1,0).
- Each segment is divided into three equal parts.
- Two new points are introduced at one-third and two-thirds along the segment.
- The middle section is replaced with two new segments, forming a 60-degree peak.
- To determine the peak's coordinates, I applied a rotation matrix, which rotates the middle segment by 60 degrees anticlockwise.
- The new points replace the original segment, and the process repeats for multiple iterations, leading to the emergence of the fractal pattern.

Results and Visualisation:



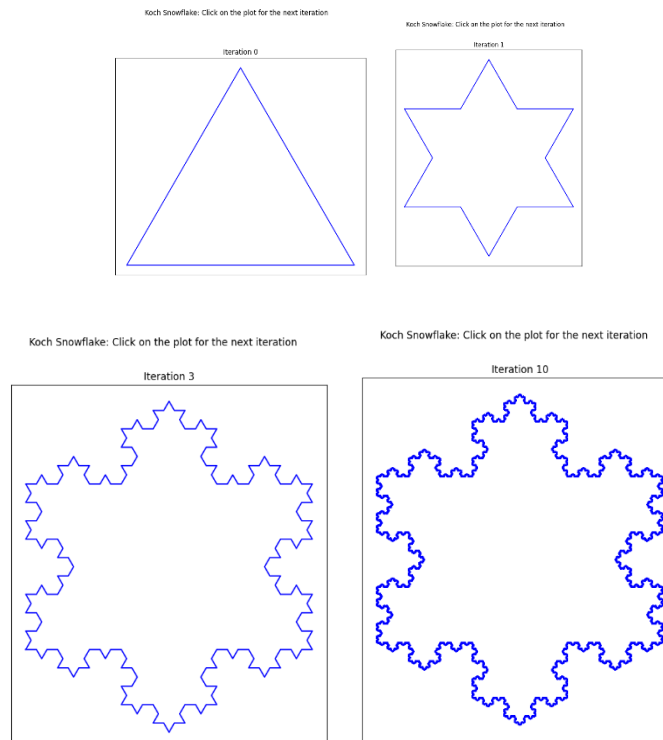
2- Koch Snowflake

General Idea: I implemented an iterative approach to generate the Koch snowflake, an extension of the Koch curve. The Koch snowflake is created by applying the Koch curve transformation to each side of an equilateral triangle rather than a single line segment. The process follows a recursive subdivision where each segment is divided into three equal parts, and the middle segment is replaced by two new segments forming a 60-degree outward triangular bump. This recursive transformation introduces self-similarity, a fundamental property of fractals.

Algorithm and Approach:

- The base shape is an equilateral triangle, defined by three points and a closing point to form a loop.
- Each side of the triangle is divided into three equal parts.
- Two new points are introduced at one-third and two-thirds along each segment.
- The middle section is replaced with two new segments forming a 60-degree outward peak.
- To compute the peak's coordinates, I applied a rotation matrix, which rotates the middle segment by -60 degrees counterclockwise to ensure that the bump is constructed outward.
- The new points replace each segment, and the process repeats for multiple iterations, leading to the emergence of the fractal snowflake.

Results and Visualisation:



3- Dragon Curve

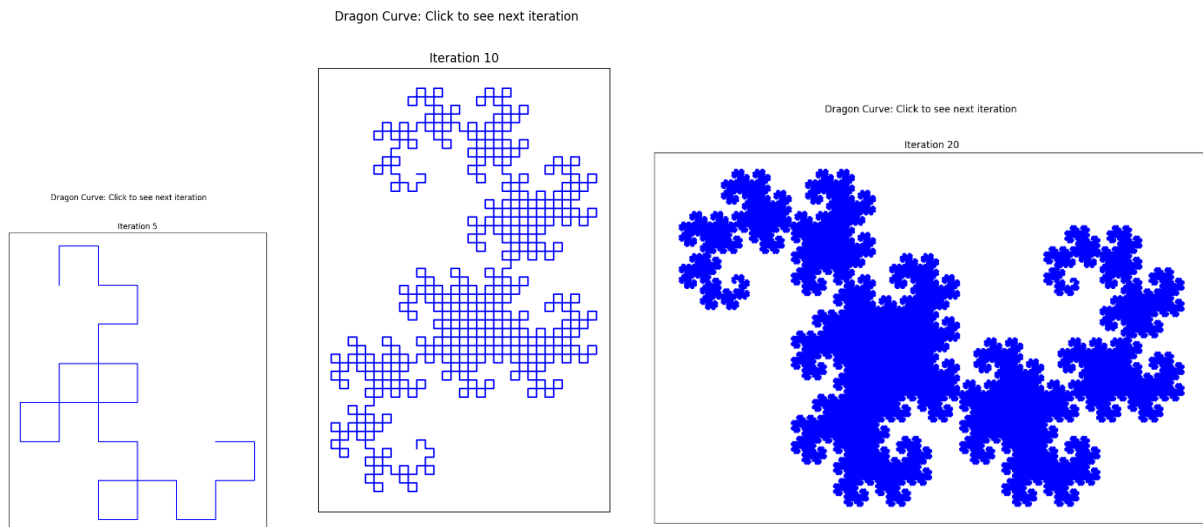
General Idea: I implemented an iterative method to generate the Dragon Curve, a well-known fractal that emerges from a simple folding pattern. The curve is constructed by repeatedly modifying a straight-line segment following a specific turning rule: at each step, the sequence of turns is mirrored and reversed, with alternating 90-degree rotations. This results in a self-similar, non-intersecting curve that becomes increasingly intricate with each iteration.

Unlike other fractals that rely on geometric transformations like scaling or rotation matrices, the Dragon Curve is purely defined by a sequence of turns. With each iteration, the number of segments doubles, and the curve reveals its distinctive self-avoiding, space-filling nature.

Algorithm and Approach:

- The curve starts as a single line segment from (0,0) to (1,0).
- The sequence of turns is updated according to the Dragon Curve rule: The current turn sequence is mirrored, reversed, and negated, with a 90-degree turn inserted in the middle.
- The new points are computed based on these turns, starting from the last point of the previous iteration.
- Each new segment is positioned by adjusting the angle accordingly.
- The process repeats iteratively, doubling the number of segments in each step, forming the characteristic Dragon Curve shape.

Results and Visualisation:



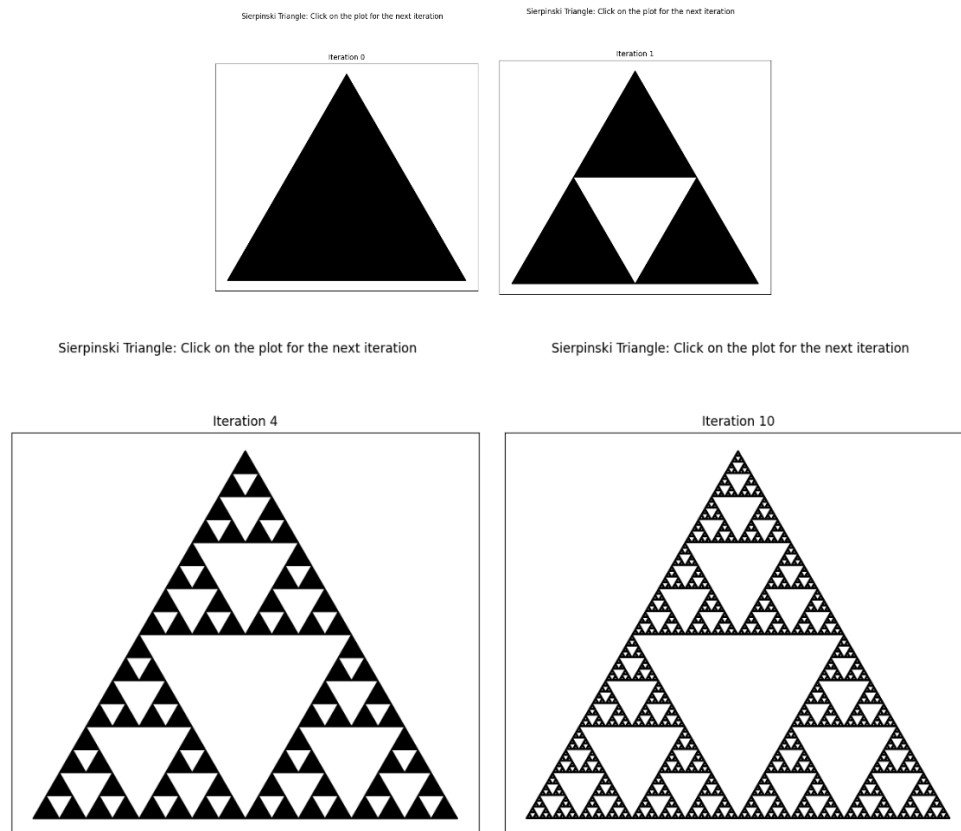
4- Sierpinski Triangle

General Idea: I implemented an iterative method to generate the Sierpinski Triangle, a classic fractal structure. The Sierpinski Triangle is formed by recursively subdividing an equilateral triangle into three smaller equilateral triangles at each iteration, removing the central area. With each iteration, the fractal grows in complexity while maintaining self-similarity, a fundamental characteristic of fractals.

Algorithm and Approach:

- The fractal starts as a single equilateral triangle.
- Each triangle is subdivided into three smaller equilateral triangles, using three transformation rules: One triangle remains at the bottom-left. One triangle is translated right. One triangle is translated upward to maintain the equilateral shape.
- The shrinking factor is 0.5, ensuring the sub-triangles are half the original size.
- The transformations are applied recursively, replacing each triangle with three smaller triangles at every iteration.
- This process repeats for multiple iterations, leading to an increasingly complex fractal.

Results and Visualisation:



5- Khayyam-Pascal Triangle

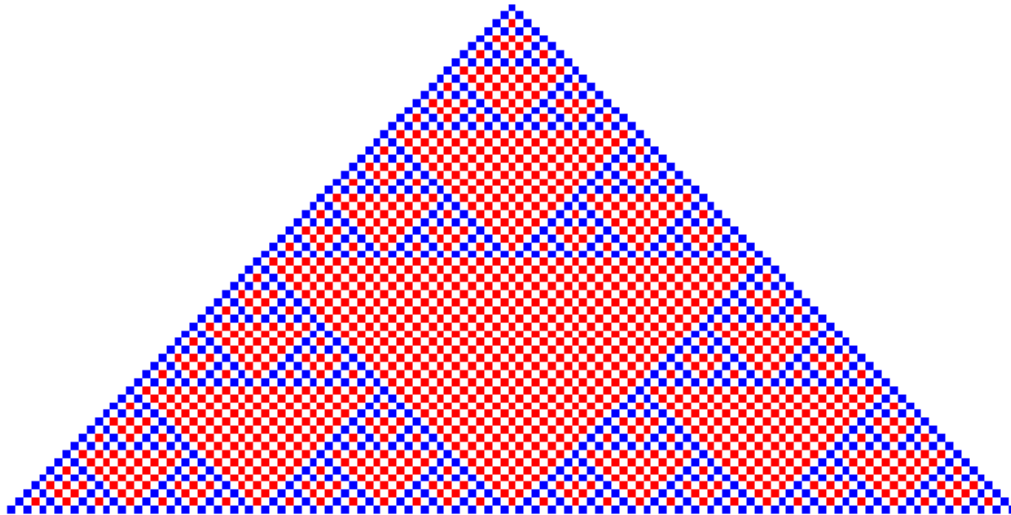
General Idea: I implemented a numerical and graphical approach to generate Pascal's Triangle and visualize its properties. Pascal's Triangle is constructed iteratively, where each number is the sum of the two numbers directly above it. Beyond its mathematical properties, this script visually highlights the distribution of odd and even numbers using a color-coded representation: Odd numbers are displayed in blue. Even numbers are displayed in red.

Algorithm and Approach:

- The triangle is initialized with a single row containing 1.
- Each subsequent row is constructed by summing adjacent elements from the previous row.
- The first and last elements of each row are always 1.
- The result is stored as a list of lists, where each sublist represents a row.
- Numbers are coloured based on parity,

Results and Visualisation:

Pascal's Triangle: Odd in Blue, Even in Red



Connection to the Sierpinski Triangle: An interesting property of Pascal's Triangle, when visualised with modulo 2 colouring, is that it closely resembles the Sierpinski Triangle. This happens because, in Pascal's Triangle, the binomial coefficients follow modular arithmetic rules, and numbers modulo 2 form a repeating self-similar pattern. As the number of rows increases, the visualisation converges toward the characteristic triangular fractal structure of Sierpinski's Triangle.

6- Sierpinski Triangle (Random)

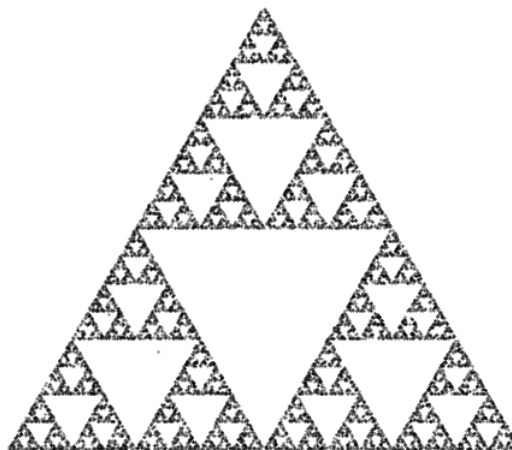
General Idea: I implemented a stochastic (randomised) approach to generate the Sierpinski Triangle, using a method known as the Chaos Game. Instead of recursively subdividing an equilateral triangle, this method randomly moves a point toward one of the triangle's vertices in each iteration. By repeating this process many times, a pattern emerges that resembles the Sierpinski Triangle, despite the randomness of point selection.

Algorithm and Approach:

- **Defining the Triangle** – The three fixed vertices of an equilateral triangle are defined as reference points.
- **Choosing a Random Initial Point** – A point is initialised at a random location inside or outside the triangle.
- In each step, one of the three vertices is chosen randomly.
- The current point moves halfway toward the chosen vertex and the new point is stored.
- This process repeats thousands of times, and the set of points gradually forms the Sierpinski Triangle structure.

Results and Visualisation:

Sierpinski Triangle (Random)



Although the method appears random, the underlying rule (moving halfway toward a randomly chosen vertex) leads to a highly structured result. The reason is that each new point is always constrained by the previous one, limiting its possible locations, any point that does not fall in a removed region continues participating in the process, reinforcing the self-similar pattern. The gaps that emerge correspond to regions that never receive points, following the self-replicating nature of the Sierpinski Triangle. Thus, despite using randomness, the process produces a highly ordered fractal structure, revealing a connection between chaos and deterministic fractals.

7- Barnsley Fern

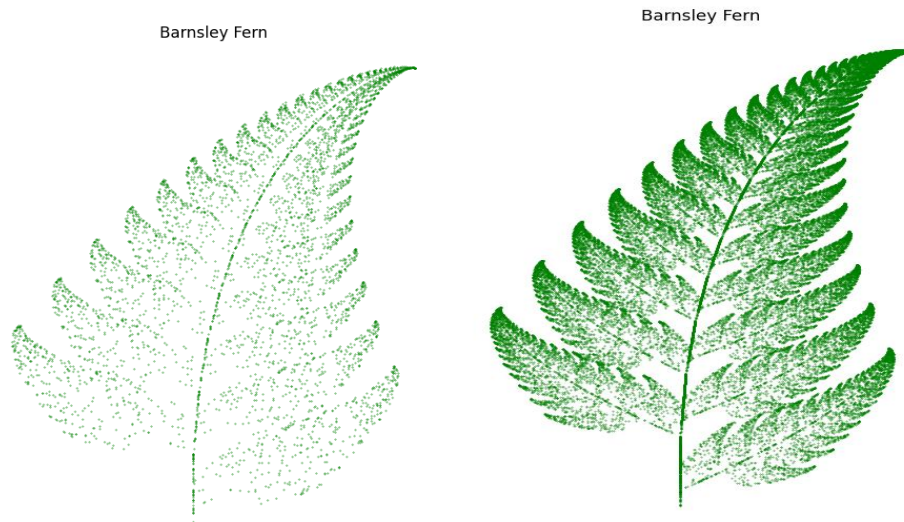
General Idea: I implemented an iterated function system (IFS) to generate the Barnsley Fern, a fractal that mimics the shape of a real fern. Unlike fractals that rely on self-replication via geometric transformations (e.g., the Sierpinski Triangle), the Barnsley Fern is produced using four different affine transformations, each applied randomly based on predefined probabilities.

By repeatedly applying these transformations, the script generates a set of points that collectively form the characteristic fern shape.

Algorithm and Approach:

- The fern consists of four transformation matrices, each representing a different structural component. Stem Transformation (1% probability) – Keeps points near the base. Main Leaf Transformation (85% probability) – Represents the overall growth structure. Left Leaflet Transformation (7% probability) – Produces smaller leaflets on the left. Right Leaflet Transformation (7% probability) – Produces smaller leaflets on the right. (The probabilities sum to 1, ensuring a valid random selection.)
- The script starts with an initial point at (0,0). At each iteration, a random number is generated to determine which transformation is applied. The selected transformation updates the point's coordinates.

Results and Visualisation:



Despite the randomness in selecting transformations, the process follows a deterministic pattern due to the way probabilities are assigned. The fern's shape emerges naturally because most points follow the main leaf structure (85%), reinforcing the dominant shape. Occasional small leaflet transformations (7%) introduce fine details. The base stem (1%) ensures the structure begins correctly. This combination of randomness and structured transformation mimics the growth of natural ferns, illustrating how fractals can model real-world organic patterns.

8- Julia Set

General Idea: I implemented an iterative method to generate and visualise the Julia Set, a complex fractal that arises from iterating a function in the complex plane. Unlike previous fractals, which rely on geometric transformations, the Julia Set is defined by the recurrence relation:

$$Z_{n+1} = Z_n^2 + c$$

Where c is a complex parameter that defines the shape of the fractal. Each point in the complex plane is iteratively updated, and the number of iterations before it escapes beyond a certain threshold ($|Z| > 2$) determines its colour in the final visualisation.

This script generates four different Julia Sets, each defined by a different complex parameter c , and saves them as high-resolution images.

Algorithm and Approach:

- A 2D grid of complex numbers is created using NumPy's meshgrid function.
- Each grid point represents a complex number Z , initialised as $Z = X + iY$, where X and Y are real and imaginary components.
- Each point Z undergoes the transformation $Z = Z^2 + c$.
- If $|Z|$ exceeds 2, the iteration stops for that point.
- A 2D array tracks the number of iterations before each point diverges.

The Julia Set is generated by iterating the function $Z_{n+1} = Z_n^2 + c$ where each point in the complex plane is tested to determine whether it escapes to infinity or remains bounded. The number of iterations before a point escapes defines its colour in the visualisation. The structure of the Julia Set is highly sensitive to the choice of c , and different values of this parameter result in vastly different fractal shapes. Some values of c produce connected Julia Sets, where a large region of the plane remains bounded, while others lead to highly fragmented sets with many points escaping quickly.

In images like number 3, where most of the fractal appears bright, the chosen value of c has resulted in a Julia Set where many points take a large number of iterations before they escape, or they do not escape at all. Since the colormap assigns brighter colours to points that persist for more iterations, the prevalence of light tones indicates that the majority of the plotted points belong to the filled Julia Set, meaning they remain within a certain bound and do not diverge to infinity. This creates an image dominated by lighter shades, as opposed to one where most points escape rapidly and are assigned darker colours. The self-similar patterns that emerge demonstrate the fractal nature of the set, where smaller structures repeat themselves within the larger formation, highlighting the balance between chaos and order in complex dynamical systems.

Results and Visualisation:

